

deploying your code. You can create custom workflows using simple declarative syntax, and GitHub provides a wide range of prebuilt actions to streamline your processes.

GitHub Actions offers a powerful way to automate your software development pipeline. Table 1 lists the key roles it plays.

Role of GitHub Actions	What it automates
• Continuous integration (CI)	Automates testing and code integration to ensure quality before merging.
• Continuous deployment (CD)	Streamlines and automates application deployments across various environments.
• Automated testing	Executes unit, integration, and end-to-end tests on every code update.
• Code linting and formatting	Automatically checks and enforces consistent code style and formatting.
• Dependency management	Handles automatic updates and version control of project dependencies.
• Security scanning	Detects and flags vulnerabilities in code automatically.
• Issue and pull request management	Automates tasks like labelling and commenting on issues and pull requests.
• Containerisation	Builds and deploys Docker containers in an automated manner.
• Custom workflows	Enables the automation of any repetitive or manual task with custom workflows.

Table 1: Key roles of GitHub Actions

How Docker Hub fits into continuous deployment

To set up automated image builds using Docker Hub and facilitate continuous deployment, you can link your GitHub repository with Docker Hub. Here's a simple example of how you can do this.

Create a new repository

- Log into your Docker Hub account and create a new repository by clicking the 'Create Repository' button.

- Provide a name for your repository (e.g., *myapp*) and select whether it should be public or private.

Link your GitHub repository

- In the repository settings, locate the 'Builds' section and click on 'Configure Automated Builds'.
- You will be prompted to connect your GitHub account. Authorise Docker Hub to access your GitHub repositories.
- Select the repository you want to link (e.g., *username/myapp*).

Specify the branch

- In the automated builds configuration, you can specify which branch to monitor (e.g., *main* or *develop*).
- When code is pushed to this branch, Docker Hub will automatically build a new image.
- Once set up, every push to the linked branch will trigger Docker Hub to build and tag a new image automatically.
- For example, if a developer pushes changes to the *main* branch, Docker Hub will execute the build process based on the Dockerfile in the repository.

Here's an example of a simple Dockerfile for a Node.js server:

```
# Dockerfile
FROM node:14

# Set the working directory
WORKDIR /app

# Copy package.json and install dependencies
COPY package.json ./
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the port
EXPOSE 3000

# Start the application
CMD ["npm", "start"]
```

To deploy the new Docker image automatically upon receiving a webhook from Docker Hub, you can set up a simple Node.js server to listen for the webhook:

```
// server.js
const express = require('express');
const { exec } = require('child_process');

const app = express();
```